

3η Σειρά Ασκήσεων
Γρηγόρης Δελατόλας (03119202)
Γιώργος Κυριάκος (03119204)

Ασκήση 1

;; We will keep the 45-sec counter in the D-E register pair
; D corresponds to decimals and E to units

START:

IN 10H ; no protection memory
LXI B,FFFFH ; for DELB

CLEAN_DISPLAY:

MVI A,10H ; 10H -> blank in 7-segment display
STA 08B0H ; 1st digit of 7-segment display
STA 00B1H ; 2nd digit of 7-segment display
STA 08B2H ; 3rd digit of 7-segment display
STA 08B3H ; 4th digit of 7-segment display
STA 08B4H ; 5th digit of 7-segment display
STA 08B5H ; 6th digit of 7-segment display
LXI D,08B0H ; for STDM
CALL STDM
CALL DCD

ENABLE_INTERRUPTS:

MVI A,0DH ; initialize interrupt mask
SIM ; RST_6.5
EI ; and enable it

WAIT_FOR_INTR:

; wait until an interruption occurs
JMP WAIT_FOR_INTR

INTR_ROUTINE:

; 8085 default name
LXI B,0FFFH ; for DELB

EI ; enable interrupts again 8085 disables them
; by default

MVI D,04H
MVI E,05H ; DE = 45
JMP DISPLAY

LEDS_ON:

MVI A,00H
STA 3000H
CALL DELB
MVI A,FFH
STA 3000H
CALL DELB

```

DISPLAY:
    MOV A,D
    STA 08B5H           ; 6th digit has the decimal
    MOV A,E
    STA 08B4H           ; 5th digit has the unit
    PUSH D              ; save D because is needed
    LXI D,08B0H
    CALL STDH           ; for the 7-segment display
    CALL DCD
    POP D               ; take D back
    MOV A,E

CHECK:
    CPI 00H             ; if unit is not zero then
    JNZ NEXT_SECOND2    ; count next second
    MOV A,D             ; if unit is zero then we
    CPI 00H             ; must check the decimal
    JZ WAIT_FOR_INTR    ; if decimale is zero then
                        ; the counting has finished

NEXT_SECOND1:
    MVI E,09H           ; else, make unit 9 and
    DCR D               ; decrease the decimal
    JMP LEDS_ON

NEXT_SECOND2:
    DCR E               ; E <- E-1 for next second
    JMP LEDS_ON         ; if E is not 0, count next second

END

```

Ασκηση 2

```
START:
    IN 10H          ; no protection memory

CLEAN_DISPLAY1:
    MVI A,10H       ; 10H -> blank in 7-segment display
    STA 08B0H       ; 1st digit of 7-segment display
    STA 00B1H       ; 2nd digit of 7-segment display
    STA 08B2H       ; 3rd digit of 7-segment display
    STA 08B3H       ; 4th digit of 7-segment display
    STA 08B4H       ; 5th digit of 7-segment display
    STA 08B5H       ; 6th digit of 7-segment display
    LXI D,08B0H     ; for STD
    CALL STD
    CALL DCD

ENABLE_INTERRUPTS:
    MVI A,0DH       ; initialize interrupt mask
    SIM             ; RST_6.5
    EI              ; and enable it

WAIT_FOR_INTR:
    LXI D,08B0H
    CALL STD
    CALL DCD
    JMP WAIT_FOR_INTR

INTR_ROUTINE:
    EI
```

```

CLEAN_DISPLAY2:
    MVI A,FFH           ; turn off reverse logic leds
    STA 3000H
    MVI A,10H           ; 10H -> blank in 7-segment display
    STA 08B0H           ; 1st digit of 7-segment display
    STA 00B1H           ; 2nd digit of 7-segment display
    STA 08B2H           ; 3rd digit of 7-segment display
    STA 08B3H           ; 4th digit of 7-segment display
    STA 08B4H           ; 5th digit of 7-segment display
    STA 08B5H           ; 6th digit of 7-segment display
    LXI D,08B0H         ; for STDM
    CALL STDM
    CALL DCD

K_INIT:
    MVI C,0AH           ; C = 10D (K1)
    MVI D,64H           ; D = 100D (K2)
    MVI E,C8H           ; E = 200D (K3)

READ_INPUT:
    CALL KIND

;RST 1

    STA 08B3H           ; 4th digit of 7-segment display

    CALL KIND
    STA 08B2H           ; 3rd digit of 7-segment display

```

```

TWO_DIGIT_HEX:
    LDA 08B3H          ; most significant hex digit
    RLC                ; shift it left 4 times
    RLC
    RLC
    RLC

    MOV B,A
    LDA 08B2H          ; least significant hex digit

    ADD B              ; A is a 2-digit hex number
    MOV B,A            ; B <- A

COMPARISON:
THRESHOLD1:
    CMP C
    JNC THRESHOLD2     ; if A >= C jump to threshold 2
    MVI A,01H          ; else the 2-digit hex number belongs
                        ; to the first threshold
    JMP OUTPUT

THRESHOLD2:
    CMP D
    JNC THRESHOLD3     ; if A >= D jump to threshold 3
    MVI A,02H          ; else the 2-digit hex number belongs
                        ; to the second threshold
    JMP OUTPUT

```

```

THRESHOLD3:
    CMP E
    JNC THRESHOLD4    ; if A >= E jump to threshold 4
    MVI A,04H         ; else the 2-digit hex number belongs
                        ; to the third threshold
    JMP OUTPUT

THRESHOLD4:
    MVI A,08H

OUTPUT:
    CMA               ; reverse logic leds
    STA 3000H         ; led on
    LXI D,08B0H
    CALL STDH
    CALL DCD          ; 7-segment display
    ;EI              ; enable interrupts again
    JMP WAIT_FOR_INTR ; wait for the next interrupt

END

```

Θεωρητικές Ασκήσεις

3η Άσκηση

α)

INR 16 MACRO ADDR

PUSH PSW ; save A
PUSH H ; save HL

LDA ADDR
INR A ; $A \leftarrow A + 1$
STA ADDR

LXI H,ADDR

INX H ; $HL \leftarrow HL + 1$
MOV A,M
ACI 0 ; add with carry
MOV M,A

POP H
POP PSW

ENDM

β)

FILL MACRO ADDR, K

PUSH PSW ; save A
PUSH H ; save H,L

LXI H, ADDR
MOV A,K

START:

MOV M,A
CPI A,01H ; if A = 1
JZ EXIT ; exit

```
        DCR A           ; else A ← A-1

        INX H           ; HL ← HL + 1
        JMP START       ; save the next integer in the next memory position
```

EXIT:

```
        POP H
        POP PSW
        ENDM
```

υ)

RHLL MACRO Q,R

PUSH PSW

```
MOV A,R
RAL           ; rotate with CY
MOV R,A
```

```
MOV A,Q
RAL           ; rotate with CY
MOV Q,A
```

POP PSW

ENDM

4η Άσκηση

Σύμφωνα με την παράγραφο 6.1.5 όταν πραγματοποιείται μία διακοπή ολοκληρώνεται αρχικά η εντολή την οποία εκτελούσε ο μΕ τη στιγμή που ενεργοποιήθηκε η είσοδος διακοπής. Άρα, εκτελείται αρχικά η εντολή JMP 2200H.

Έπειτα, απενεργοποιούνται όλες οι διακοπές, και σώζεται στη στοίβα η κατάσταση του μΕ. Με άλλα λόγια, σώζεται ο μετρητής προγράμματος και ο καταχωρητής σημαιών. Άρα, ο SP μειώνεται κατά 4 θέσεις και γίνεται 2FFCH.

Στη συνέχεια, σύμφωνα με την παράγραφο 6.2.1 για την RST 6.5 ο PC παίρνει την τιμή 0034H και εκτελείται μία εντολή άλματος για να μεταφερθεί ο PC στη κατάλληλη θέση για την ρουτίνα εξυπηρέτησης της διακοπής. Εάν η ρουτίνα εξυπηρέτησης χρησιμοποιεί κάποιον καταχωρητή τον σώζει πρώτα στη στοίβα. Άρα, ο SP μειώνεται ανάλογα με τον αριθμό των καταχωρητών που χρησιμοποιεί η ρουτίνα εξυπηρέτησης της διακοπής.

Στο τέλος της ρουτίνας εξυπηρέτησης ανακτάται η κατάσταση του μΕ με την ανάκτηση (κατά αντίστροφη σειρά) όλων καταχωρητών σώθηκαν στην αρχή, και ενεργοποιούνται όλες οι διακοπές.

Μετά το πέρας της διακοπής ο SP θα είναι 3000H και ο PC 2200H.

5η Άσκηση

MVI A,0EH

SIM

LXI H,0

MVI C,32D

MVI B,00H

; B is used in order to find out whether the device sends
; MSB's or LSB's

EI

; interrupts activation

ADDR:

MOV A,C

; check if all numbers have been sent

CPI 0

JNZ ADDR

DI

; disable interrupts

DAD H

; calculation of mean by adding HL 4 times with itself

DAD H

DAD H

DAD H

DAD H

; HL is left shifted 4 times

; mean is stored in H

RST 5.5:

PUSH PSW

; store A in stack

MOV A,B

; check if the device sends LSB's (B = 0) or MSB's (B = 1)

CPI 01H

JZ FOR_MSB

FOR_LSB:

IN 20H

RRC

RRC

RRC

RRC

; make four RRC in order to place the bits at the
; 4 LSB positions

ANI 0FH

; isolate only the LSB's

MVI B,01H

; next bits will be MSB's

JMP RETURN

FOR_MSB:

IN 20H

```
ANI F0H          ; isolate only the MSB;s  
MVI B,00H        ; next bits will be LSB's
```

RETURN:

```
MVI D,00H  
MOV E,A  
DAD D  
DCR C  
POP PSW  
EI  
RET
```

β)

```
LXI H,00H
MVI C,32D
MVI B,00H                ; B is used in order to find out whether the device sends
                          ; MSB's or LSB's
```

```
START:
    IN 20H
    ANI 01H
    CPI 00H
    JZ NEW_DATA          ; If x0=0 jump to NEW_DATA
    JMP START
```

```
NEW_DATA:
    IN 20H
    ANI 01H
    CPI 00H
    JZ NEW_DATA
```

```
NEW_DATA_2:                ; We are sure that x0=1 , the data has been transmitted

    MOV A,B                ; check if the device sends LSB's (B = 0) or MSB's (B = 1)
    CPI 01H
    JZ FOR_MSB
```

```
FOR_LSB:
    IN 20H
    RRC
    RRC
    RRC
    RRC                    ; make four RRC in order to place the bits at the
                          ; 4 LSB positions
    ANI 0FH                ; isolate only the LSB's
    MVI B,01H              ; next bits will be MSB's
    JMP RETURN
```

```
FOR_MSB:
    IN 20H
    ANI F0H                ; isolate only the MSB's
    MVI B,00H              ; next bits will be LSB's
```

RETURN:

MVI D,00H

MOV E,A

DAD D

DCR C

MOV A,C

; check if all numbers have been sent

CPI 00H

JNZ START

DAD H

; calculation of mean by adding HL 4 times with itself

DAD H

DAD H

DAD H

; HL is left shifted 4 times

; mean is stored in H

END